

Synchronous FIFO Design Using Verilog HDL

RTL Design, Simulation and Verification

Prepared By

Priya Nageswari Karanam

Engineering Design Report

Table of Contents

1. Project Overview	- -	1
2. FIFO Fundamentals	- -	1
(a) What is a FIFO?		
(b) Synchronous FIFO		
(c) Applications		
3. System Architecture	- -	2
(a) FIFO Block Diagram		
(b) FIFO Memory Organization		
(c) Write Operation		
(d) Read Operation		
(e) Full and Empty Flag Logic		
4. RTL Design and Implementation	- -	4
(a) Complete RTL Code		
5. RTL Design Explanation	- -	6
(a) Module Interface		
(b) Internal Registers and Memory Organization		
(c) Write and Read Enable Logic		
(d) Memory Access Logic		
(e) Next-State Logic		
(f) Sequential State Update Logic		
6. Verification Environment	- -	11
(a) Complete Testbench Code		
7. Verification Results	- -	12
(a) RTL Schematic Analysis		
(b) Behavioral Simulation Results		
8. Future Enhancements	- -	14
9. Conclusion	- -	14

1 Project Overview

A First-In-First-Out (FIFO) memory is one of the most widely used buffering techniques in digital systems for temporary data storage and transfer. FIFO structures are commonly employed whenever data produced by one module must be stored and retrieved in the same order by another module.

This project presents the design and verification of an 8×8 Synchronous FIFO using Verilog HDL. The FIFO is capable of storing eight 8-bit data words and supports controlled read and write operations using a single clock signal. The design incorporates write and read pointer management, full and empty status flag generation, and protection against overflow and underflow conditions.

The RTL design was developed using Verilog HDL and verified through extensive simulation and testing. Vivado was used for RTL development, simulation, and schematic generation. Various test scenarios, including FIFO filling, FIFO emptying, overflow prevention, underflow prevention, and simultaneous read/write operations, were performed to validate the correctness of the design.

The objective of this project is to gain practical experience in memory-based digital design, pointer control mechanisms, status flag generation, and functional verification methodologies that are widely used in FPGA and ASIC design flows.

2 FIFO Fundamentals

2.1 What is a FIFO?

FIFO (First-In-First-Out) is a memory structure that stores and retrieves data in the order in which it was received. The first data element written into the FIFO is the first element to be read out, making it suitable for temporary data buffering applications.

A FIFO consists of a memory array along with control logic that manages data storage and retrieval operations. FIFO memories are widely used in digital systems to facilitate reliable data transfer between different functional modules while preserving the sequence of data.

For example, if the data sequence 11, 22, and 33 is written into a FIFO, the same sequence will be obtained during read operations, ensuring ordered data transfer.

2.2 Synchronous FIFO

A Synchronous FIFO is a FIFO memory in which both read and write operations are controlled by the same clock signal. Since all operations occur with respect to a single clock, the design is simpler and easier to implement compared to an Asynchronous FIFO.

In the proposed design, the write pointer, read pointer, memory array, and status flag logic are synchronized using a common clock. This ensures predictable data transfer and eliminates clock domain synchronization issues.

Synchronous FIFOs are commonly used in digital systems where the data producer and data consumer operate at the same clock frequency. Their simple architecture and reliable operation make them suitable for FPGA and ASIC-based designs.

2.3 Applications of FIFO

FIFO memories are widely used in digital systems wherever temporary data buffering is required. They help in storing data produced by one module and delivering it to another module in the same order.

Common applications of FIFO include:

- Data buffering in communication systems.
- Processor and memory interfacing.
- UART, SPI, and other serial communication protocols.
- Streaming and multimedia data transfer.
- FPGA and ASIC-based digital designs.

The ability of a FIFO to maintain data order while supporting independent read and write operations makes it an essential component in modern digital systems.

3 System Architecture

The Synchronous FIFO consists of a memory array, write and read control logic, pointer management circuitry, and status flag generation logic. These components work together to ensure ordered data storage and retrieval while preventing overflow and underflow conditions.

3.1 FIFO Block Diagram

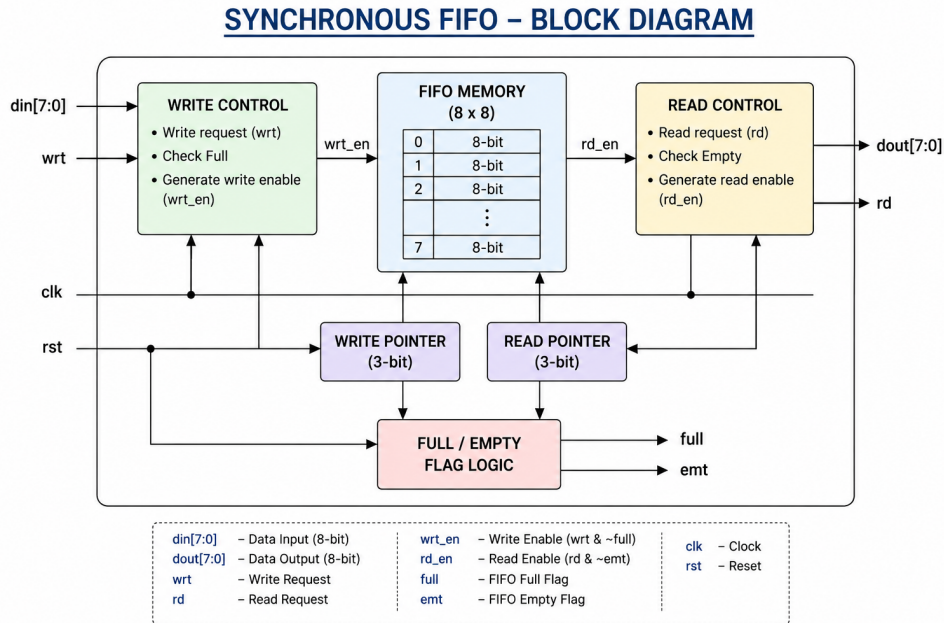


Figure 1: Block Diagram of the Synchronous FIFO

The FIFO architecture consists of five major components: FIFO Memory, Write Control, Read Control, Pointer Logic, and Status Flag Generation. The Write Control block stores incoming data into the FIFO memory whenever a valid write request is received. Similarly, the Read Control block retrieves data from the FIFO memory when a read request is asserted.

The Pointer Logic maintains the locations of the next write and read operations using write and read pointers. The Status Flag Generation block continuously monitors the FIFO status and generates the Full and Empty flags, preventing invalid write and read operations.

3.2 FIFO Memory Organization

The proposed FIFO uses an 8×8 memory organization, meaning that the FIFO can store up to eight data words, with each word consisting of 8 bits. The memory is implemented as a register array in Verilog HDL and serves as the storage element of the FIFO.

The write pointer identifies the location where the next incoming data word will be stored, while the read pointer identifies the location from which the next data word will be retrieved. As data is written into or read from the FIFO, the corresponding pointers are incremented to access the subsequent memory locations.

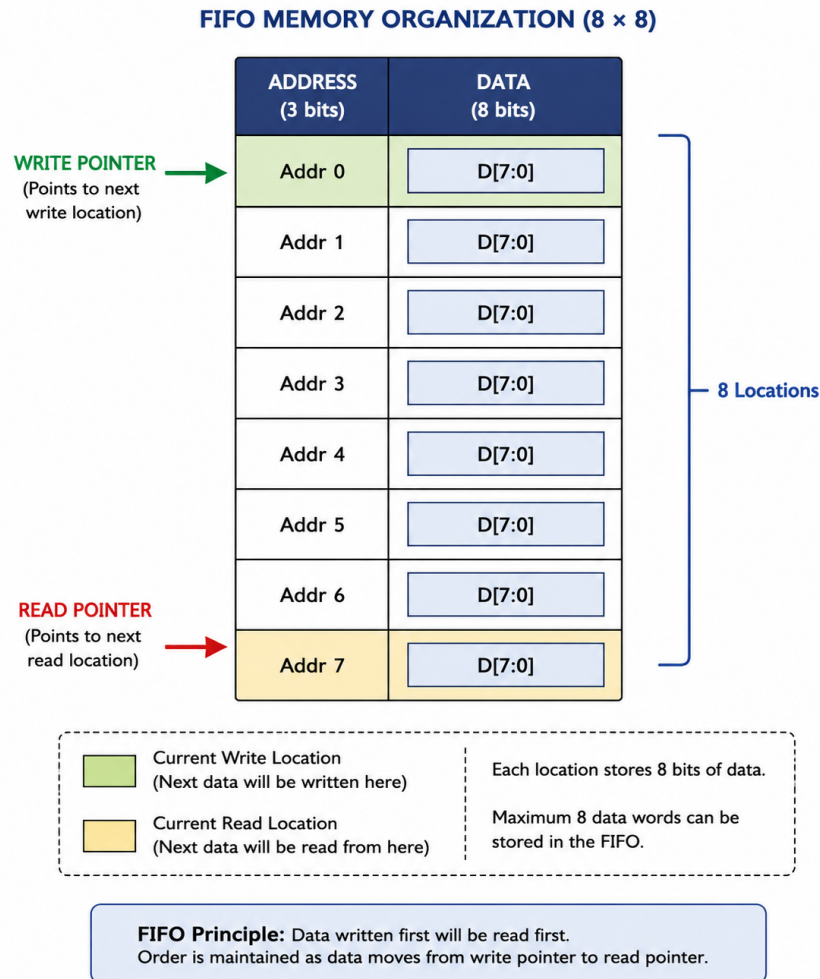


Figure 2: FIFO Memory Organization (8×8)

3.3 Write Operation

The write operation stores incoming data into the memory location indicated by the write pointer whenever `wrt` is asserted and the FIFO is not full. After a successful write operation, the write pointer advances to the next location. If the FIFO becomes full, additional write operations are blocked to prevent overflow.

3.4 Read Operation

The read operation retrieves data from the memory location indicated by the read pointer whenever `rd` is asserted and the FIFO is not empty. After a successful read operation, the read pointer advances to the next location. If the FIFO becomes empty, additional read operations are blocked to prevent underflow.

3.5 Full and Empty Flag Logic

The Full and Empty flags are used to indicate the current status of the FIFO and to prevent invalid operations.

The Empty flag is asserted when there is no valid data available for reading from the FIFO memory. In this condition, read operations are disabled until new data is written into the FIFO. This prevents underflow conditions and ensures that invalid data is not transferred to the output.

The Full flag is asserted when all memory locations in the FIFO have been occupied. Once the FIFO becomes full, additional write operations are blocked until space becomes available through a read operation. This prevents overflow conditions and protects the stored data from being overwritten.

The Full and Empty flags are generated using the positions of the write and read pointers. These flags prevent invalid write and read operations, ensuring reliable FIFO functionality.

4 RTL Design and Implementation

This section presents the complete Verilog HDL implementation of the Synchronous FIFO. The design includes memory organization, write and read control logic, pointer management circuitry, and status flag generation required for reliable FIFO operation.

4.1 Complete RTL Code

```
module FIFO(input clk,rst,wrt,rd, input [7:0]din, output full,emt,output reg [7:0]dout);
    reg [7:0] fifo [0:7];
    reg [2:0] wrt_ptr, wrt_ptr_nxt, wrt_ptr_inc;
    reg [2:0] rd_ptr, rd_ptr_nxt, rd_ptr_inc;
    reg fifo_full_reg, fifo_emt_reg, full_nxt,emt_nxt;

    //writing into memory
    wire wrt_en,rd_en;
```

```

assign wrt_en = (wrt & ~fifo_full_reg);
assign rd_en = (rd & ~fifo_emt_reg);
always @ (posedge clk) begin
    if(wrt_en)
        fifo[wrt_ptr] <= din;
    if(rd_en)
        dout <= fifo[rd_ptr];
end

//reading from memory
assign full = fifo_full_reg;
assign emt = fifo_emt_reg;

//calculation part or nxt process
always @ * begin
    // increment process
    wrt_ptr_inc = wrt_ptr + 1;
    rd_ptr_inc = rd_ptr + 1;
    // holding state
    wrt_ptr_nxt = wrt_ptr; //wrt_ptr is where to store [0 -> 7]
    rd_ptr_nxt = rd_ptr; // rd_ptr is from where to take [7 -> 0]
    full_nxt = fifo_full_reg;
    emt_nxt = fifo_emt_reg;
    case({wrt,rd})
        2'b00 : begin end
        2'b10 : begin
            if(~fifo_full_reg) begin
                wrt_ptr_nxt = wrt_ptr_inc;
                emt_nxt = 0;
                if(wrt_ptr_inc == rd_ptr)
                    full_nxt = 1;
            end end
        2'b01 : begin
            if(~fifo_emt_reg) begin
                rd_ptr_nxt = rd_ptr_inc;
                full_nxt = 0;
                if(rd_ptr_inc == wrt_ptr)
                    emt_nxt = 1;
            end
        end
    endcase
end

```

```

                end end
            2'b11 : begin
                if(~fifo_full_reg && ~fifo_emt_reg) begin
                    wrt_ptr_nxt = wrt_ptr_inc;
                    rd_ptr_nxt  = rd_ptr_inc;
                end end
        endcase
end
always @(posedge clk or posedge rst) begin    // posedge rst
    if(rst) begin
        wrt_ptr <= 0;
        rd_ptr <= 0;
        fifo_full_reg <= 0;
        fifo_emt_reg <= 1;
    end
    else begin
        wrt_ptr <= wrt_ptr_nxt;
        rd_ptr <= rd_ptr_nxt;
        fifo_full_reg <= full_nxt;
        fifo_emt_reg <= emt_nxt;
    end
end
end
endmodule

```

5 RTL Design Explanation

This section provides a detailed explanation of the Verilog HDL implementation of the Synchronous FIFO. The design consists of memory storage elements, control logic, pointer management circuitry, and status flag generation logic. Each part of the RTL design contributes to the reliable storage and retrieval of data while maintaining FIFO functionality.

5.1 Module Interface

The FIFO module serves as the top-level design entity and defines all input and output signals required for FIFO operation. The module accepts clock, reset, write, and read control signals along with an 8-bit input data bus. It generates an 8-bit output data bus and status flags indicating whether the FIFO is full or empty.

The module declaration is shown below:

```
module FIFO(input clk, rst, wrt, rd, input [7:0] din, output full, emt, output reg [7:0] dout );
```


The input and output signals are summarized below:

- **clk**: System clock signal.
- **rst**: Reset signal used to initialize the FIFO.
- **wrt**: Write request signal.
- **rd**: Read request signal.
- **din[7:0]**: Input data bus.
- **dout[7:0]**: Output data bus.
- **full**: Indicates that the FIFO is full.
- **emt**: Indicates that the FIFO is empty.

5.2 Internal Registers and Memory Organization

The FIFO design utilizes a memory array along with several internal registers to manage data storage, pointer movement, and status flag generation.

The memory array is declared as `reg [7:0] fifo[0:7]`. This creates eight memory locations, each capable of storing an 8-bit data word, resulting in a total storage capacity of 8×8 bits.

The write pointer registers (`wrt_ptr`, `wrt_ptr_inc`, and `wrt_ptr_nxt`) are responsible for tracking the current write location, calculating the incremented pointer value, and storing the next-state write pointer respectively.

Similarly, the read pointer registers (`rd_ptr`, `rd_ptr_inc`, and `rd_ptr_nxt`) track the current read location, generate the incremented read pointer value, and store the next-state read pointer.

The FIFO status is monitored using the registers `fifo_full_reg` and `fifo_emt_reg`, which indicate whether the FIFO is full or empty. The signals `full_nxt` and `emt_nxt` are used to calculate the next-state values of these flags before they are updated on the active clock edge.

Together, these memory elements and control registers form the internal architecture of the FIFO and enable reliable data storage, retrieval, and status monitoring.

5.3 Write and Read Enable Logic

The FIFO uses write and read enable signals to determine whether memory access operations are permitted. The corresponding RTL code is shown below:

```
wire wrt_en, rd_en;
assign wrt_en = (wrt & ~fifo_full_reg);
assign rd_en  = (rd & ~fifo_emt_reg);
```

The signal `wrt_en` is asserted only when a write request (`wrt`) is received and the FIFO is not full. Similarly, the signal `rd_en` is asserted only when a read request (`rd`) is received and the FIFO is not empty.

If the FIFO is full, `wrt_en` remains low, preventing additional data from being written into the memory. Likewise, if the FIFO is empty, `rd_en` remains low, preventing invalid read operations.

These enable signals act as protection mechanisms against overflow and underflow conditions and are used by the memory access logic to control actual data transfers.

5.4 Memory Access Logic

The Memory Access Logic is responsible for performing actual data storage and retrieval operations within the FIFO memory. The corresponding RTL implementation is shown below:

```
always @ (posedge clk) begin
    if(wrt_en)
        fifo[wrt_ptr] <= din;

    if(rd_en)
        dout <= fifo[rd_ptr];
end
```

This block is triggered on every positive edge of the clock signal. When the write enable signal (`wrt_en`) is asserted, the input data (`din`) is stored in the memory location indicated by the write pointer (`wrt_ptr`).

Similarly, when the read enable signal (`rd_en`) is asserted, the data stored at the memory location pointed to by the read pointer (`rd_ptr`) is transferred to the output register (`dout`).

Since both operations are synchronized with the clock signal, the FIFO provides reliable and predictable data transfer. This block forms the core data path of the FIFO, enabling ordered storage and retrieval of information.

5.5 Next-State Logic

The Next-State Logic is responsible for calculating the future values of the write pointer, read pointer, Full flag, and Empty flag. This logic is implemented using a combinational always block, as shown below:

```
always @ * begin
    wrt_ptr_inc = wrt_ptr + 1;
    rd_ptr_inc = rd_ptr + 1;
    wrt_ptr_nxt = wrt_ptr;
    rd_ptr_nxt = rd_ptr;
    full_nxt = fifo_full_reg;
    emt_nxt = fifo_emt_reg;
    case({wrt,rd})    ...    endcase
end
```

The incremented pointer values are first calculated and stored in `wrt_ptr_inc` and `rd_ptr_inc`. These values are later used to determine the next write and read locations.

The next-state registers are initially assigned their current values. This ensures that the FIFO remains in its current state unless a valid read or write operation occurs.

A case statement is then used to examine the combination of write and read requests. Based on the values of `wrt` and `rd`, the logic determines how the pointers and status flags should be updated for the next clock cycle.

This block acts as the decision-making unit of the FIFO and generates all next-state values before they are transferred into the sequential registers.

5.5.1 No Operation Condition

The no-operation condition occurs when both the write and read requests are inactive. The corresponding RTL code is shown below:

```
2'b00 : begin end.
```

In this situation, neither data storage nor data retrieval takes place. Consequently, the write pointer, read pointer, Full flag, and Empty flag retain their current values.

This condition allows the FIFO to maintain its existing state until a valid write or read request is received.

5.5.2 Write Operation Condition

The write operation condition occurs when the write request is active and the read request is inactive. The corresponding RTL implementation is shown below:

```
2'b10 : begin
    if(~fifo_full_reg) begin
        wrt_ptr_nxt = wrt_ptr_inc;      emt_nxt = 0;
        if(wrt_ptr_inc == rd_ptr)
            full_nxt = 1;
    end end
```

When a valid write request is received and the FIFO is not full, the write pointer is incremented to indicate the next available memory location. Since new data is being stored, the Empty flag is cleared.

The logic also checks whether the incremented write pointer becomes equal to the current read pointer. If this condition occurs, all FIFO locations have been occupied and the Full flag is asserted.

This logic ensures proper pointer advancement while preventing overflow conditions.

5.5.3 Read Operation Condition

The read operation condition occurs when the read request is active and the write request is inactive. The corresponding RTL implementation is shown below:

```
2'b01 : begin
    if(~fifo_emt_reg) begin
```

```

    rd_ptr_nxt = rd_ptr_inc;    full_nxt = 0;
    if(rd_ptr_inc == wrt_ptr)
        emt_nxt = 1;
end    end

```

When a valid read request is received and the FIFO is not empty, the read pointer is incremented to access the next memory location. Since data is being removed from the FIFO, the Full flag is cleared.

The logic then checks whether the incremented read pointer becomes equal to the current write pointer. If this condition occurs, all stored data has been read and the Empty flag is asserted.

This mechanism ensures correct data retrieval while preventing underflow conditions.

5.5.4 Simultaneous Read and Write Condition

The simultaneous read and write condition occurs when both write and read requests are active during the same clock cycle. The corresponding RTL implementation is shown below:

```

2'b11 : begin
    if(~fifo_full_reg && ~fifo_emt_reg) begin
        wrt_ptr_nxt = wrt_ptr_inc;
        rd_ptr_nxt  = rd_ptr_inc;
    end    end

```

In this condition, a new data word is written into the FIFO while an existing data word is read from the FIFO at the same time. Therefore, both the write pointer and read pointer are incremented simultaneously.

Since one data word enters the FIFO and one data word leaves the FIFO during the same clock cycle, the overall occupancy of the FIFO remains unchanged. As a result, the Full and Empty flags typically retain their current status.

This capability allows continuous data flow through the FIFO and improves data throughput in high-speed digital systems.

5.6 Sequential State Update Logic

The Sequential State Update Logic is responsible for updating the FIFO registers on the active edge of the clock signal. It also initializes the FIFO to a known state whenever the reset signal is asserted. The corresponding RTL implementation is shown below:

```

always @(posedge clk or posedge rst) begin
    if(rst) begin
        wrt_ptr <= 0;
        rd_ptr <= 0;
        fifo_full_reg <= 0;
        fifo_emt_reg <= 1;
    end
end

```

```

else begin
    wrt_ptr <= wrt_ptr_nxt;
    rd_ptr <= rd_ptr_nxt;
    fifo_full_reg <= full_nxt;
    fifo_emt_reg <= emt_nxt;
end
end
end

```

When the reset signal is asserted, both the write and read pointers are initialized to zero. The Full flag is cleared, and the Empty flag is asserted to indicate that the FIFO contains no valid data.

During normal operation, the next-state values generated by the combinational logic are transferred into the corresponding registers on each positive clock edge. This includes updating the write pointer, read pointer, Full flag, and Empty flag.

By separating the next-state calculations from the register updates, the design follows a structured finite-state-machine style approach, improving readability and simplifying verification.

6 Verification Environment

To verify the functionality of the Synchronous FIFO, a dedicated Verilog testbench was developed. The testbench generates the clock signal, applies reset conditions, performs various read and write operations, and evaluates the FIFO under normal as well as boundary conditions.

Several test scenarios including FIFO filling, FIFO emptying, overflow protection, underflow protection, and simultaneous read/write operations were performed to validate the correctness of the design.

6.1 Complete Testbench Code

```

module FIFO_tb;
    reg clk , rst , wrt , rd ;
    reg [7:0] din ;
    wire full , emt ;
    wire [7:0] dout ;

    FIFO dut ( clk , rst , wrt , rd , din , full , emt , dout ) ;
    always #5 clk = ~clk ;
    initial begin
        $monitor ("T=%0t - rst=%0b - wrt=%0b - rd=%0b - din=%0h - dout=%0h - full=%0b - emt=%0b" ,
            $time , rst , wrt , rd , din , dout , full , emt ) ;
        clk = 0 ; rst = 0 ; din = 0 ; wrt = 0 ; rd = 0 ; #5
        rst = 1 ; #10
        rst = 0 ;
    end
endmodule

```

```

wrt = 1;
din = 8'h11; #10;    din = 8'h22; #10;
din = 8'h33; #10;    din = 8'h44; #10;
din = 8'h55; #10;    din = 8'h66; #10;
din = 8'h77; #10;    din = 8'h88; #10;
wrt = 0;

//Overflow condition
wrt = 1;    din = 8'hAA; #10;    wrt = 0;
rd = 1; #40;    rd = 0;

wrt = 1;    din = 8'h99; #10;    din = 8'hBB; #10;    wrt = 0;
rd = 1; #80;    rd = 0;

//Underflow condition
rd = 1; #10;    rd = 0;

// Simultaneous Read and Write
wrt = 1;    din = 8'hCC; #10;    wrt = 0;
wrt = 1;    rd = 1;    din = 8'hDD; #10;
wrt = 0;    rd = 0;

rd = 1; #20;    rd = 0; #20;
$finish;
end
endmodule

```

7 Verification Results

The functionality of the Synchronous FIFO was verified through simulation in Vivado. Multiple test scenarios, including normal operation, overflow protection, underflow protection, and simultaneous read/write operations, were performed to validate the correctness of the design.

7.1 RTL Schematic Analysis

The RTL schematic generated by Vivado confirms the successful synthesis of the FIFO design. The generated hardware structure matches the intended RTL architecture.

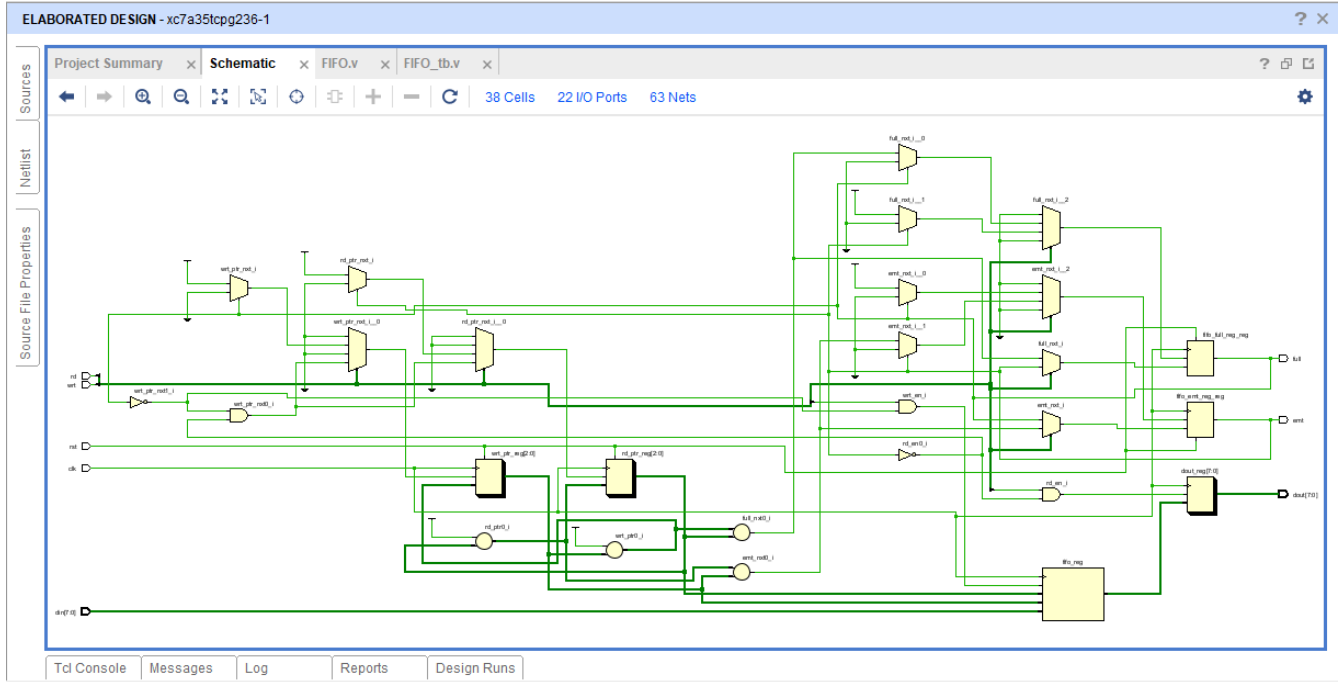


Figure 3: RTL Schematic of the Synchronous FIFO

7.2 Behavioral Simulation Results

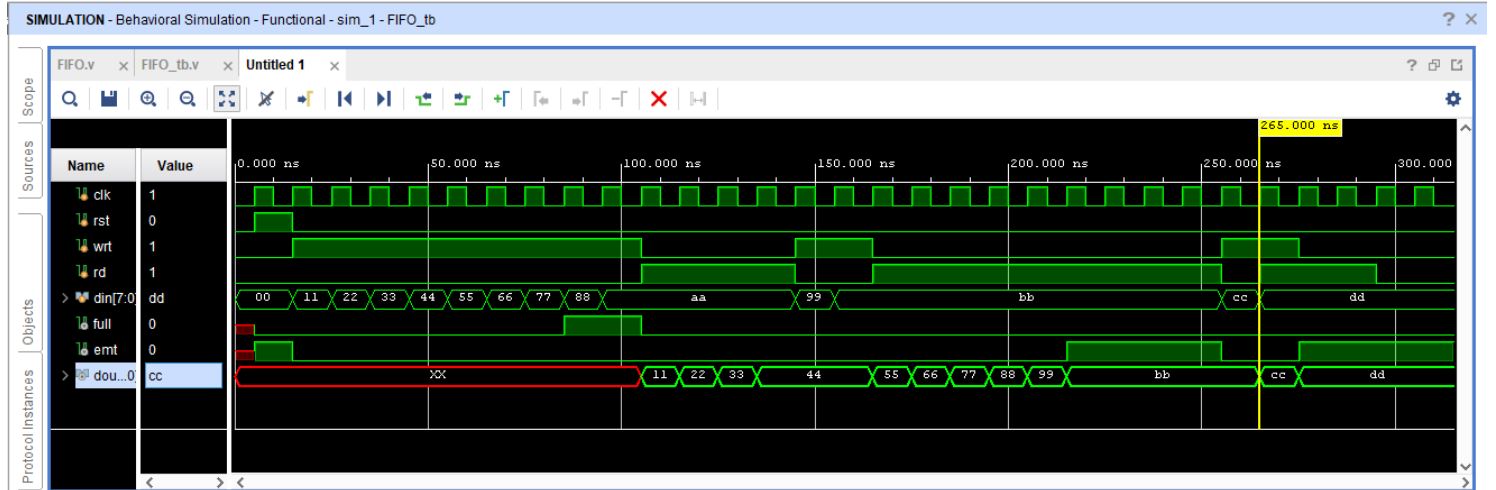


Figure 4: Behavioral Simulation of the Synchronous FIFO

The behavioral simulation was performed to verify the functional correctness of the FIFO design. The waveform confirms that data is written into and read from the FIFO in the correct First-In-First-Out order.

The simulation also verifies the proper operation of the Full and Empty status flags during data storage and retrieval. The observed behavior matches the expected FIFO functionality and confirms the correctness of the RTL implementation.

8 Future Enhancements

8.1 Increased FIFO Depth

The storage capacity of the FIFO can be increased to support larger amounts of data and improve buffering capability in high-speed applications.

8.2 Parameterized Design

The FIFO can be converted into a parameterized design, allowing the data width and memory depth to be configured according to system requirements.

8.3 Asynchronous FIFO

The current design uses a single clock for read and write operations. An asynchronous FIFO can be developed to support data transfer between different clock domains.

8.4 FPGA Implementation

The FIFO can be implemented and tested on an FPGA platform to evaluate its real-time hardware performance.

8.5 Advanced Status Flags

Additional status indicators such as Almost Full and Almost Empty flags can be incorporated to provide improved control over data flow management.

9 Conclusion

This project successfully demonstrated the design and verification of an 8×8 Synchronous FIFO using Verilog HDL. The FIFO was implemented with efficient memory management, write and read pointer control, Full and Empty status flag generation, and protection against overflow and underflow conditions.

The functionality of the design was verified through simulation, confirming correct First-In-First-Out data transfer behavior under various operating scenarios. The results validated the reliability of the FIFO architecture and its ability to perform controlled data storage and retrieval operations.

Through this project, practical experience was gained in RTL design, combinational and sequential logic implementation, memory-based digital systems, and functional verification methodologies. The concepts and techniques explored in this work form a strong foundation for the development of more advanced digital systems used in FPGA and ASIC design.